

Daily Stock Residual Return Prediction

Krystal Lin & Julius Gruber

August 26, 2024

Abstract

This white paper presents an application of practical machine learning techniques to predict the cumulative residual returns of a liquid universe of approximately 500 stocks. This paper presents an analysis of the stock market data from 2010-2014, delves into the tuning and selection of several ML models, namely Regularized Linear Regression, Random Forests, and Boosting. Each model was trained, validated, and tested to ensure robustness and best generalization to out of sample data.

1 Exploratory Data Analysis

1.1 Analysis and Visualization

1.1.1 Target Variable

At 15:30 each day, we make prediction about the cumulative residual return for the next 24 hours. The target variable can be calculated as $y = r_i^{16:00} - r_i^{15:30} + r_{i+1}^{15:30}$ where r_i^t represents the cumulative residual return for a stock at day i time t with respect to day $i - 1$'s closing.

Statistic	Value
count	62557
mean	0.000308528
std	0.0120280405
min	-0.738711730
25%	-0.0085258210
50%	-0.0002020943
75%	0.0055808163
max	1.9488992520

Table 1: Summary Statistics of Cumulative Residual Return

As we can see from the summary statistics above, the target variable is approximately symmetric around zero, but has significantly negative and positive outliers. To fit the

model, we standardized the returns based on the $t - 1$ estimated vol, and clip the data cross-sectionally at 5 MAD from the median. Figure below shows the p-p plot of the raw vs standardized and clipped target variables.

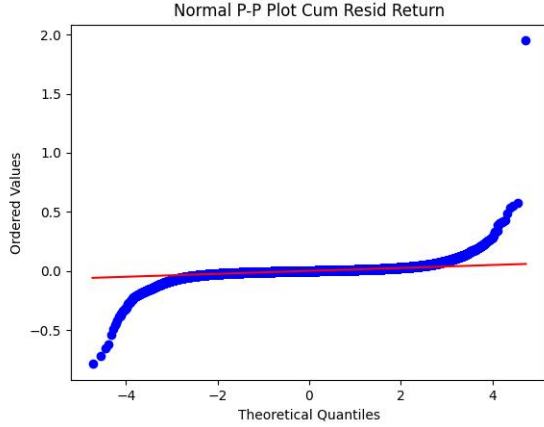


Figure 1: P-P Plot Raw Resid Return

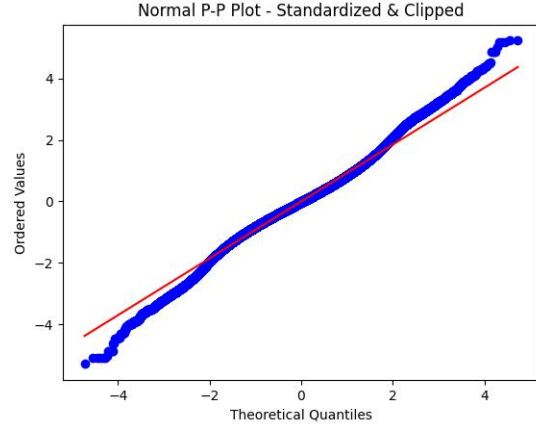
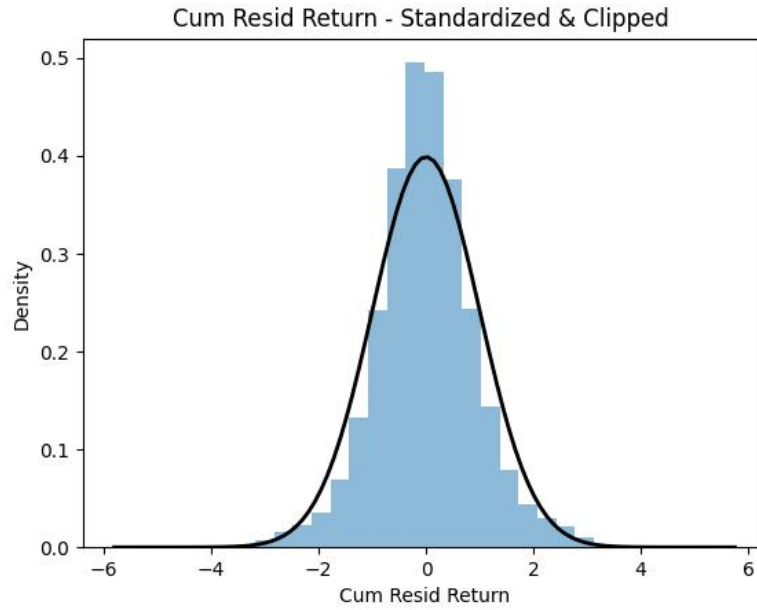


Figure 2: P-P Plot Resid Return Cleaned

Fitting on the standardized and clipped target variable makes the model less sensitive to outliers. Despite this, the histogram of the cleaned target variable shows that it continues to be leptokurtic relative to the normal. We observe higher mass near the center and samples 3 standard deviations away from the mean.



1.2 Feature Engineering

Feature	Description
Exchange	Identifies the financial exchange where the stock is traded, NYSE vs NASDAQ.
Corporate Action	Stock split or dividend issuance identified from the changes in daily price and shares adjustment factors
Volume Change Normalized	Indicates cross-sectionally normalized percentage changes in trading volume to facilitate comparison across different periods.
Cum Residual Returns at 15:30	Cumulative residual returns as of the time to make prediction.
Rolling Cum Residual Returns	Cumulative residual returns over a rolling period (5 days and 10 days), useful for identifying trends.
Relative Strength Index (RSI)	Momentum oscillator that measures speed and change of price movements, indicating overbought or oversold conditions.
Accumulation/Distribution Indicator (AD) Change	Measures the change in the cumulative flow of money into and out of a stock, indicating accumulation or distribution.

Table 2: Summary of Feature Space

1.2.1 Notes on Technical Indicators

- **Relative Strength Index (RSI):**

The Relative Strength Index (RSI) is a momentum oscillator that measures the speed and change of price movements. The RSI oscillates between 0 and 100 and is typically used to identify overbought or oversold conditions in a instrument.

Calculation: The RSI is calculated using the following formula:

$$RSI = 100 - \frac{100}{1 + RS} = 100 - \frac{100}{1 + \frac{AvgGain}{AvgLoss}} \quad (1)$$

where RS is the average gain of up periods during the specified time frame divided by the average loss of down periods. The time frame typically used is 14 periods.

Interpretation:

- An RSI above 70 indicates that a security may be overbought, suggesting a potential sell signal.
- An RSI below 30 indicates that a security may be oversold, suggesting a potential buy signal.

- **Accumulation/Distribution Indicator (AD):**

The Accumulation/Distribution Line is a volume-based indicator that measures the cumulative inflow/outflow of money into a security.

Calculation: The AD line is calculated by considering the closing price’s position within the day’s range and then multiplying this by the day’s volume.

$$AD_t = AD_{t-1} + \frac{(Close - Low) - (High - Close)}{(High - Low)} \times Volume \quad (2)$$

where AD_t is today’s Accumulation/Distribution value and AD_{t-1} is the previous day’s value. In our features, we considered daily changes in the AD line.

Interpretation:

- A rising AD line suggests accumulation and a potential upward price movement.
- A falling AD line suggests distribution and a potential downward price movement.

1.3 Data Processing

Feature	Description	Normalized	Clipped
Exchange	Identifies the financial exchange where the stock is traded, NYSE vs NASDAQ.	NA	NA
Corporate Action	Stock split or dividend issuance identified from the changes in daily price and shares adjustment factors	NA	NA
Volume Change Normalized	Measures the change in the log of Cum Value at 15:30, adjusted for shares adjustment factor.	Cross Sectional	4 Std
Cum Residual Returns at 15:30	Cumulative residual returns at 15:30. The raw data has the best validation result comparing to clipped and or normalized.	No	5 MAD Cross Sectional
Rolling Residual Returns	Rolling residual returns over 5 and 10 days, based on EOD(16:00) cum resid returns. Using the rolling returns at $t - 1$ as feature value.	$ESTVOL_{t-1}$	5 MAD Cross Sectional
Relative Strength Index (RSI)	Applied on the intraday data residual returns from 9:45 to 15:30 (Cum residual returns are diffed to obtain the raw resid return).	Cross Sectional	4 Std
AD Change	Calculated daily AD change using daily high, low, close, and volume (adjusted for shares adjustment factor), and used $t - 1$'s value as the feature.	$ESTVOL_{t-1}$ and Cross Sectional	5 MAD Cross Sectional

Table 3: Summary of Feature Space

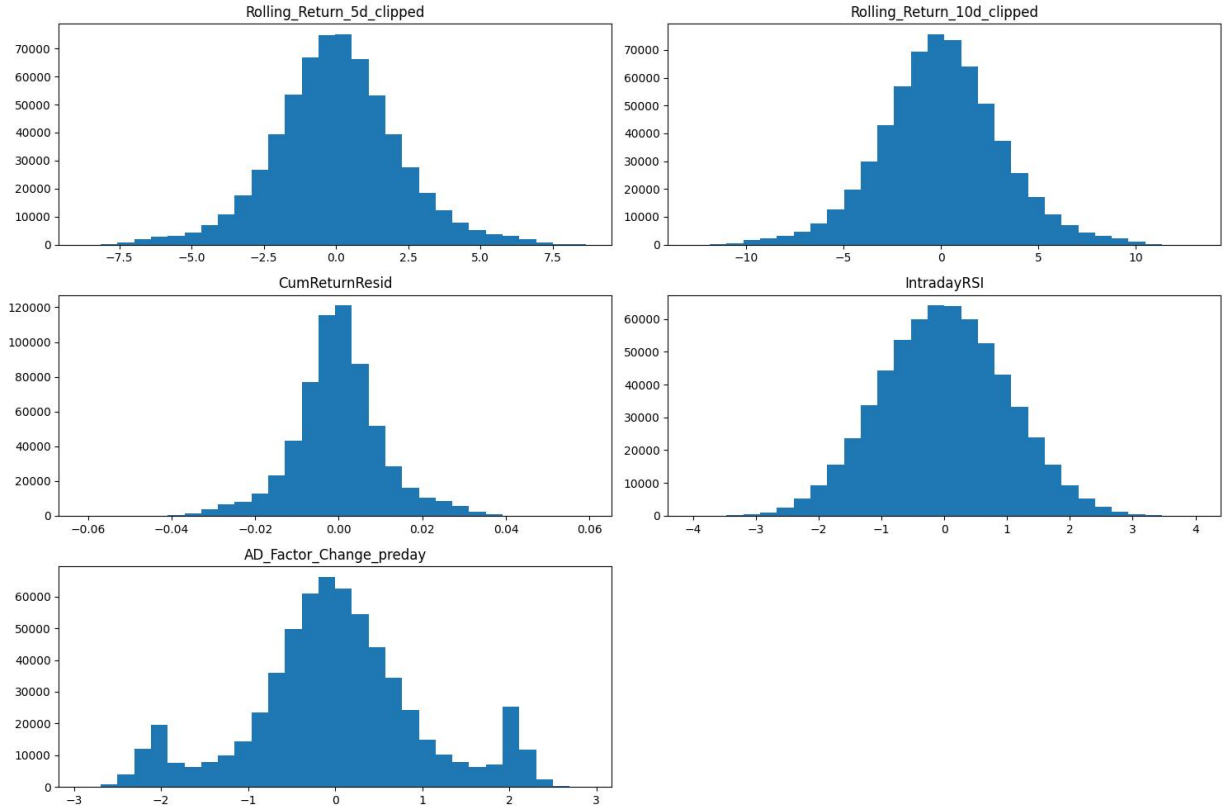


Figure 3: Feature Distributions

2 Cross-Validation Framework

The training data spans from 2010 to 2014. As the first step, the data is split into the training set (2010 to 2013) and the validation set (2014). Since the target variable is the 1 day ahead return, we included an embargo period of 1 day when performing the split. In other words, the first trading day of 2014 is discarded from the validation set.

Using the training data from 2010 to 2013, we adopted the expanding window cross validation approach. The cross-validation consists of 3 folds: (2010, 2011), (2010-2011, 2012), (2010-2012, 2013), where the first element represent the periods covered by the training set, and the second element corresponds to the period covered by the validation set. Again, an embargo period of 1 day is applied to perform the split.

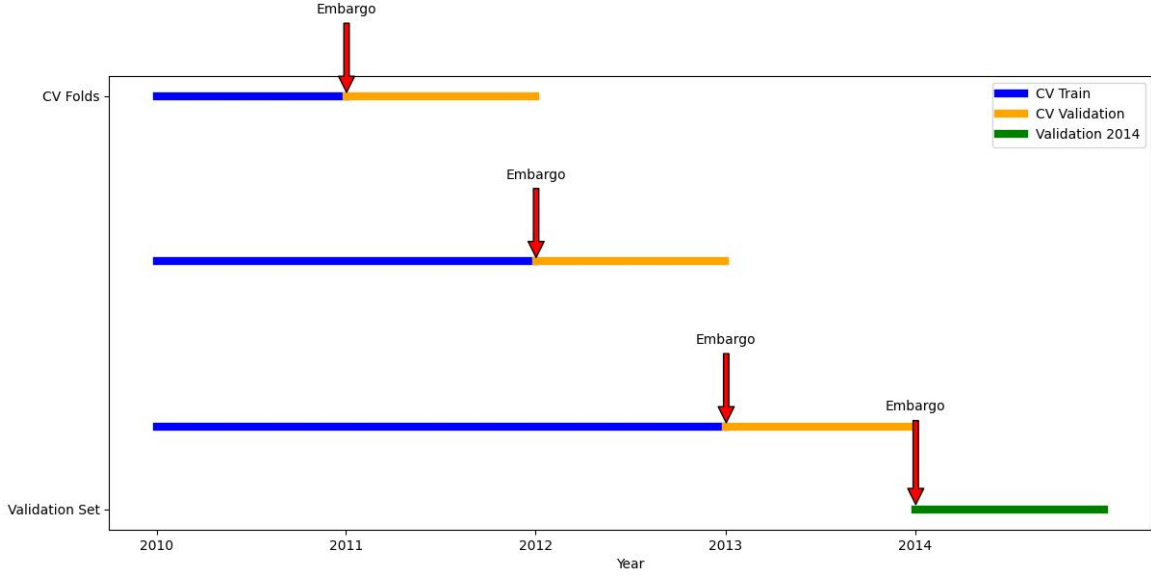


Figure 4: Cross Validation Framework

3 Model and Feature Selection

3.1 Linear Models

The use of linear models was motivated by their interpretability relative to tree models. To avoid overfitting, we tested lasso and ridge, using l_1 and l_2 penalty for the coefficients, respectively. The two figures 5 and 6 below show the regularization paths for the two models. As the the penalty increases, the coefficients are pushed towards zero. We observe that the parameters shrink at a higher speed in lasso, as ridge requires a high penalty when the coefficients are small in scale. The plots here also provide guidance on feature selection. We observe that *IntradayRSI*, *ADFactorChangePreday*, *Dividend*, *RollingReturn10dclipped* are features with high absolute coefficients in lasso. *IntradayRSI*, *ADFactorChangePreday*, *Dividend*, *RollingReturn10dclipped*, *StockSplit* are features with high absolute coefficients in ridge.

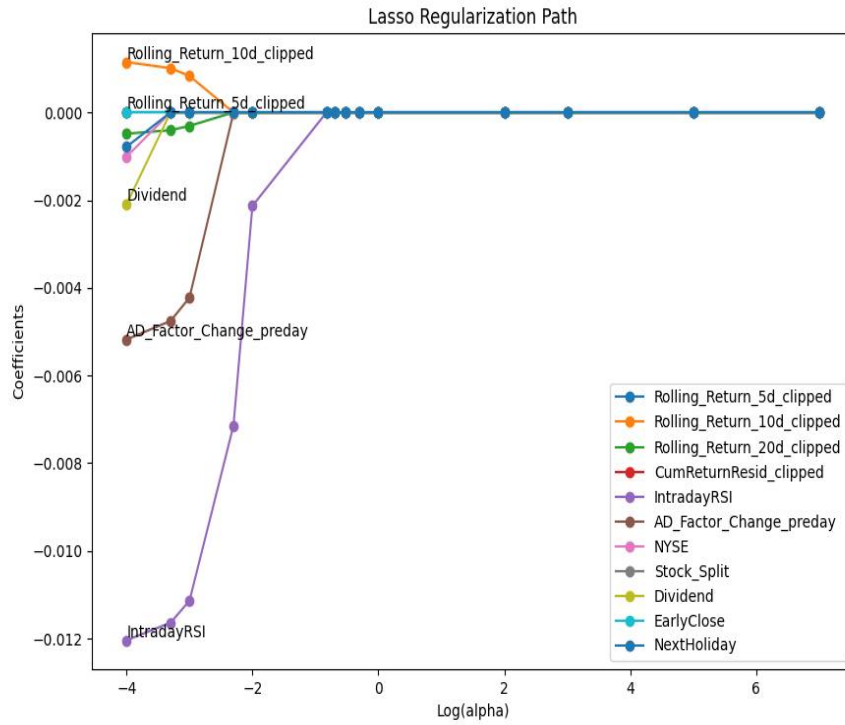


Figure 5: Lasso Regularization Path, $\alpha \in [0.0001, 100]$

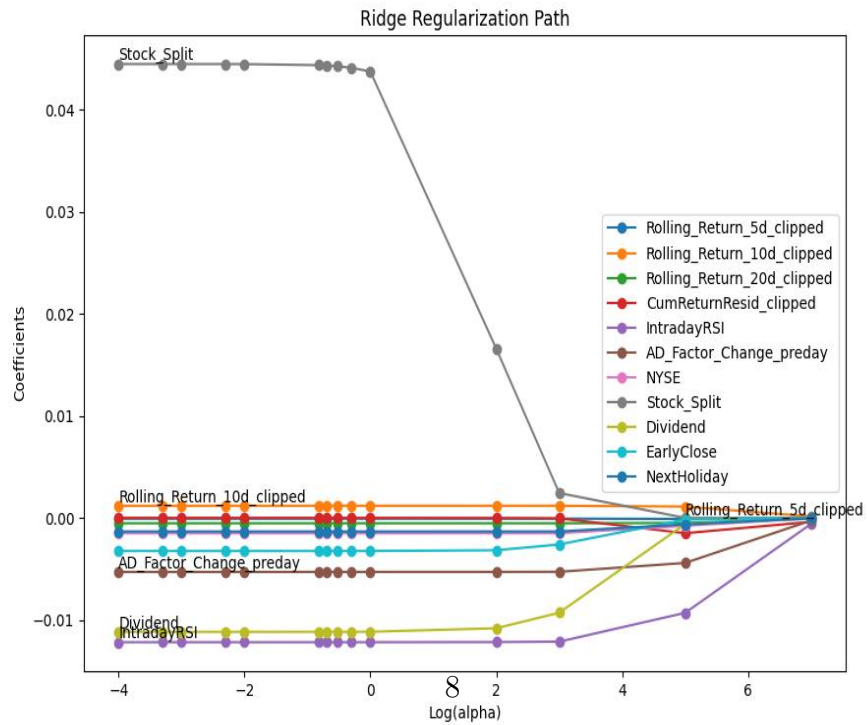


Figure 6: Ridge Regularization Path, $\alpha \in [0.0001, 100]$

3.2 Tree Based Models

Despite the interpretability of linear models, they are unable to capture the nonlinear effects of features on the target. In this section, we apply tree models, which generally learn the nonlinear and interaction effects better.

3.2.1 Parameter Fine tuning

For the hyperparameter tuning, we opted for the open-source package Optuna on our instructors' recommendation. The most significant advantage we noted was the ability to examine the importance of specific features and their variability. This insight allowed us to fine-tune the features more effectively. We utilized three different models: Random Forest, XGBoost, and LightGBM.

3.2.2 Random Forest

As a benchmark, we employed the Random Forest Regression model from scikit-learn. We chose to vary the `n_estimators`, `max_depth`, and `min_samples_leaf` parameters within a reasonable range of values.

Parameter Name	Value Range
<code>n_estimators</code>	[100, 200, 300]
<code>max_depth</code>	[2, 4, 6]
<code>min_samples_leaf</code>	[1, 3, 5, 10]

Table 4: Parameter ranges for Random Forest Regression

We are first looking at the individual hyper parameter importance for the model in 7.

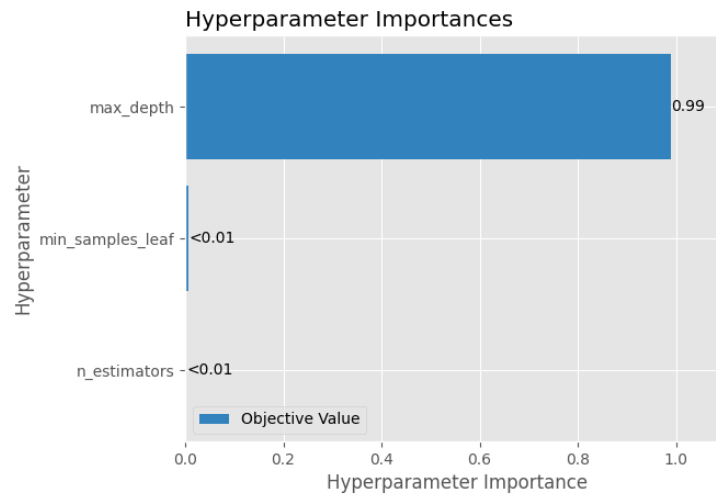


Figure 7: Hyper Parameter Importance for Random Forest

We can see that the `max_depth` feature is the most crucial one for the model, whilst the other ones do not seem to be very important. Hence we decided to look into this feature and fix and then tune the others. In 8 we see that models with `max_depth` 2 outperform the models with different ones drastically.



Figure 8: Varying of different parameters

3.2.3 XGBoost Regression

XGBoost has been one of the most used models in recent years for Tabular based data. Furthermore, it was discussed in great detail in the lectures. The table 5 depicts the various hyperparameter pairs we tried out. Similar to the Random Forest regression, we started by looking at the importance of various hyperparameters 12.

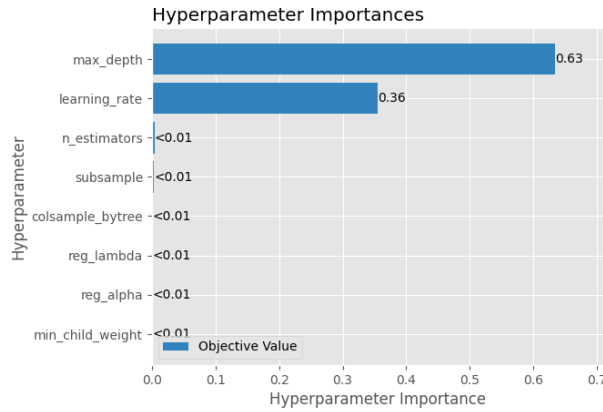


Figure 9: Hyper Parameter Importance for XGBoost

We therefore decided to look into the distribution for the max_depth and learning rate parameter

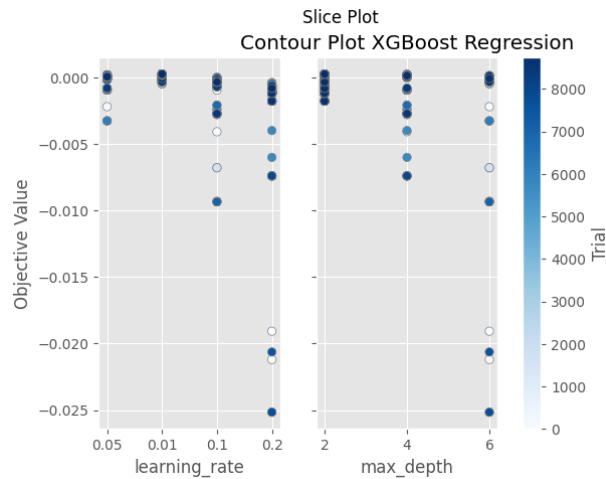


Figure 10: Varying of learning rate and max depth for XGBoost

3.2.4 Light GBM Regression

As another Tree based model we used LightGBM Regression , which got introduced in 2017. The hyper parameters are depicted in 6. As with the other two models, we started by first looking at the importance of the different hyper parameters in 11.

Parameter Name	Value Range
max_depth	[2, 4, 6]
learning_rate	[0.005, 0.01, 0.1, 0.2]
n_estimators	[100, 200, 300]
subsample	[0.7, 0.9, 1.0]
colsample_bytree	[0.5, 0.7, 1.0]
min_child_weight	[1, 5, 10]
reg_alpha	[0, 0.1, 1]
reg_lambda	[1, 1.5, 2]

Table 5: Parameter ranges for XGBoost

Parameter Name	Value Range
max_depth	[2, 4, 6]
learning_rate	[0.01, 0.1, 0.2]
n_estimators	[100, 150]
subsample	[0.7, 0.9, 1.0]
colsample_bytree	[0.5, 0.7, 1.0]
min_child_samples	[1, 5, 10]
lambda_l2	[0.05, 0.1, 0.2, 0.3]

Table 6: Parameter ranges for LightGBM

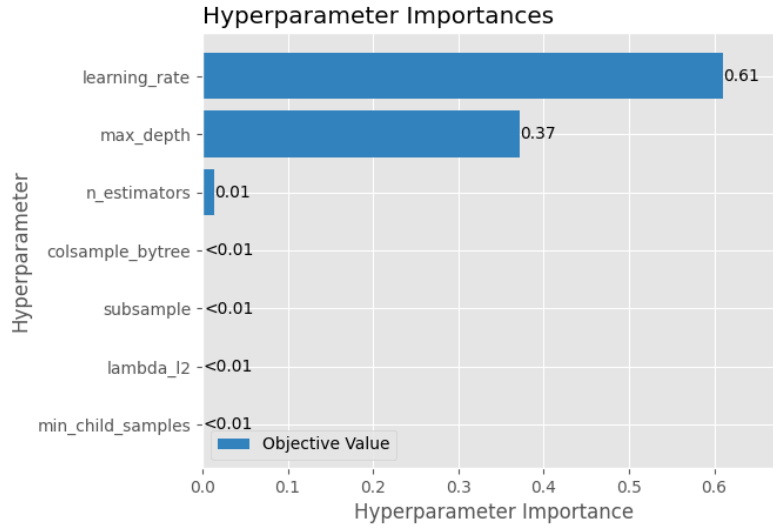


Figure 11: Varying of learning rate and max depth for LightGBM

Similar to XGboost the models seemed mainly dependend on the learning_rate and the max_depth parameter. Hence we fine-tuned our models on those two parameters.

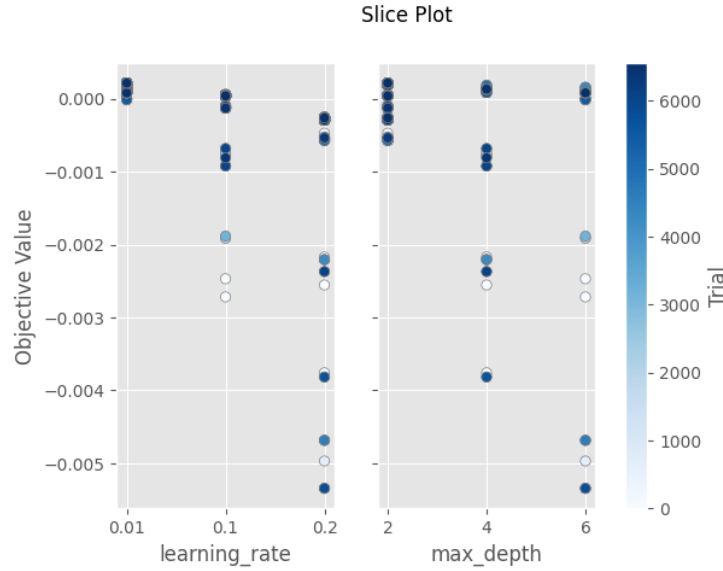


Figure 12: Varying of learning rate and max depth for LightGBM

3.3 Final Cross Validation Results

Based on the hyperparamter importances and hyperparamter optimization history from Optuna, we selected parameter grids and performed grid search with cross validation.

3.4 Validation Score

The table below summarizes the validation score for the linear and tree-based models in order of their validation R^2 . As we can see, tree-based models generally produced better result.

Table 7: Weighted R^2 on 2014 Validation Data

Model	Weighted R^2
XGBoost	0.000137
Random Forest	0.000071
LightGBM	-0.000012
Lasso	-0.000059
Ridge	-0.000136

3.4.1 Random Forest

Table 7 below presents the grid values, and the resulting feature importance from Random Forest and MDA. MDA showed consistent results as the model's feature importance. The hyper parameter configuration for the model was set as follows:

`max_depth:5, max_features:sqrt, min_samples_leaf:100, and n_estimators:150.`

Figure 11 shows the in sample and out of sample weighted R^2 as a function of the min sample leaves, with the red dot corresponding to our current hyper parameter selection. The test curve is relatively flat, but achieves a local maximum at our current selection.

Table 8: Random Forest Parameter Grid and Feature Importance

Random Forest Parameter Grid	
n_estimators	[100, 150, 200]
max_features	['sqrt', 'log2']
max_depth	[5, 10, 15]
min_samples_leaf	[20, 40, 100]

Feature	Feature Importance	MDA
CumReturnResid	0.268616	0.000270
IntradayRSI	0.247432	0.000254
AD_Factor_Change_preday	0.190142	0.000173
Rolling_Return_5d_clipped	0.146101	0.000132
Rolling_Return_10d_clipped	0.135358	0.000101
NYSE	0.04152	0.000010

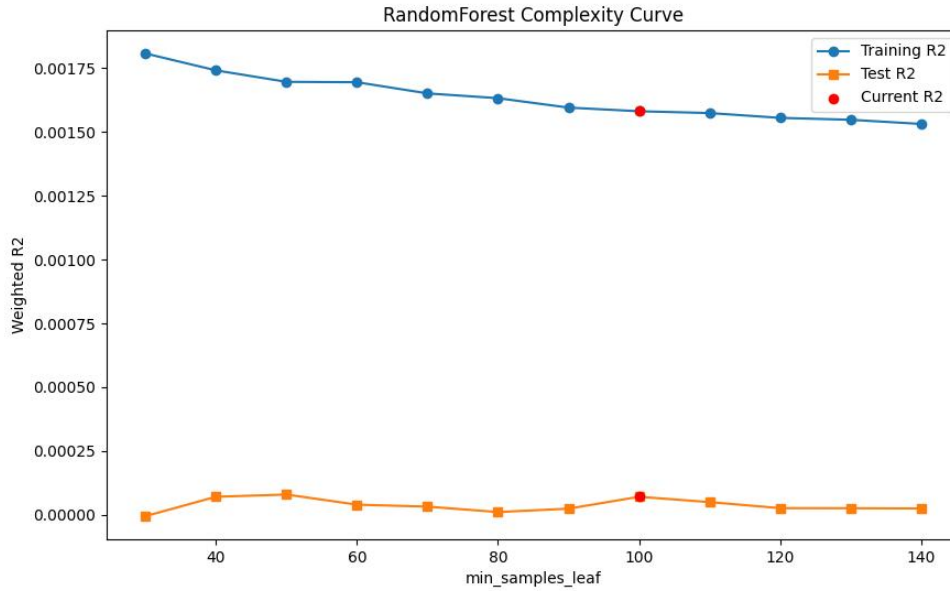


Figure 13: Random Forest Training vs Test Weighted R^2

3.4.2 XGBoost

9 below presents the grid values, and the resulting feature importance from XGBoost and MDA. Again, MDA showed consistent results with the model's feature importance. The hyper parameter configuration for the model was set as follows:

colsample_bytree: 0.8, learning_rate: 0.01, max_depth: 2, min_child_weight: 10, n_estimators: 200, reg_alpha: 0.01, and subsample: 0.7.

Figure 14 shows the in sample and out of sample weighted R^2 as a function of the learning rate/complexity. The test curve shows that the weighted R^2 achieves a local maximum as complexity approaches the current value, and decays afterwards, showing no evidence of overfitting.

Table 9: XGBoost Parameter Grid and Feature Importance

XGBoost Parameter Grid	
max_depth	[2, 4, 6]
min_child_weight	[10, 20, 30]
learning_rate	[0.01, 0.05, 0.1]
n_estimators	[100, 200]
subsample	[0.7, 0.9]
colsample_bytree	[0.6, 0.8, 1.0]
reg_alpha	[0.01, 0.1, 0.2, 0.3]

Feature	Feature Importance	MDA
IntradayRSI	0.252045	0.000168
CumReturnResid	0.212645	0.000251
Rolling_Return_5d_clipped	0.152593	0.000082
AD_Factor_Change_preday	0.150557	0.000098
Rolling_Return_10d_clipped	0.133136	0.000062
NYSE	0.099024	0.000003

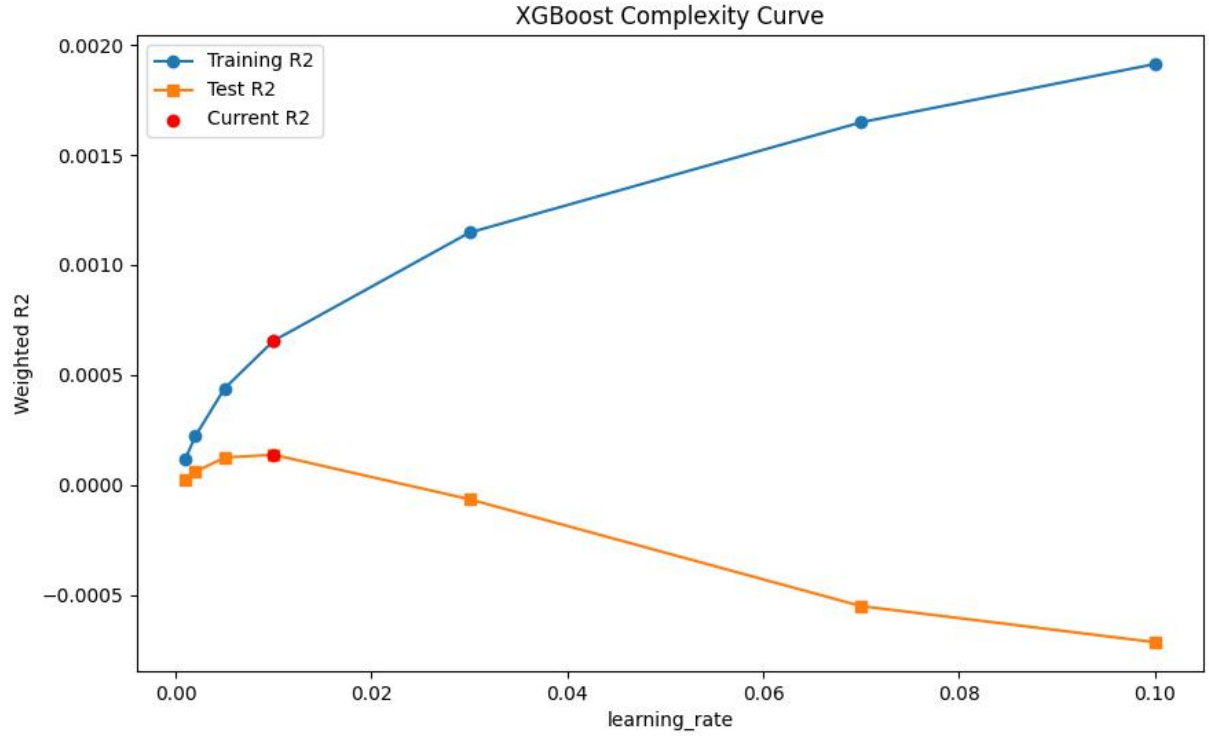


Figure 14: XGBoost Training vs Test Weighted R^2

3.4.3 LightGBM

10 below presents the grid values, and the resulting feature importance from LightGBM and MDA. Feature importance and MDA yielded slightly different results for *IntradayRSI* and *CumReturnResit* but remain consistent overall. The hyperparameter configuration for the model was set as follows:

`colsample_bytree: 1.0, lambda_l1: 50, learning_rate: 0.01, max_depth: 2,`
`min_samples_leaf: 150, n_estimators: 150, and subsample: 0.7.`

Figure 15 shows the in sample and out of sample weighted R^2 as a function of the min sample leaves, with the red dot corresponding to our current hyperparameter selection. As we can see, the test curve is relatively flat, but achieves a local maximum at our current selection.

Table 10: LightGBM Parameter Grid and Feature Importance

LightGBM Parameter Grid	
max_depth	[2, 4, 6]
learning_rate	[0.01, 0.1, 0.2]
n_estimators	[100, 150, 200]
subsample	[0.7, 0.9, 1.0]
colsample_bytree	[0.6, 0.8, 1.0]
min_samples_leaf	[50, 100, 150]
lambda_l1	[0.2, 0.3, 0.4, 1, 5, 10, 20, 30, 40, 50]

Feature	Feature Importance	MDA
CumReturnResid	214	0.000178
IntradayRSI	106	0.000299
AD_Factor_Change_preday	67	0.000065
Rolling_Return_5d_clipped	35	0.000028
Rolling_Return_10d_clipped	15	0.000017
NYSE	13	0.000008

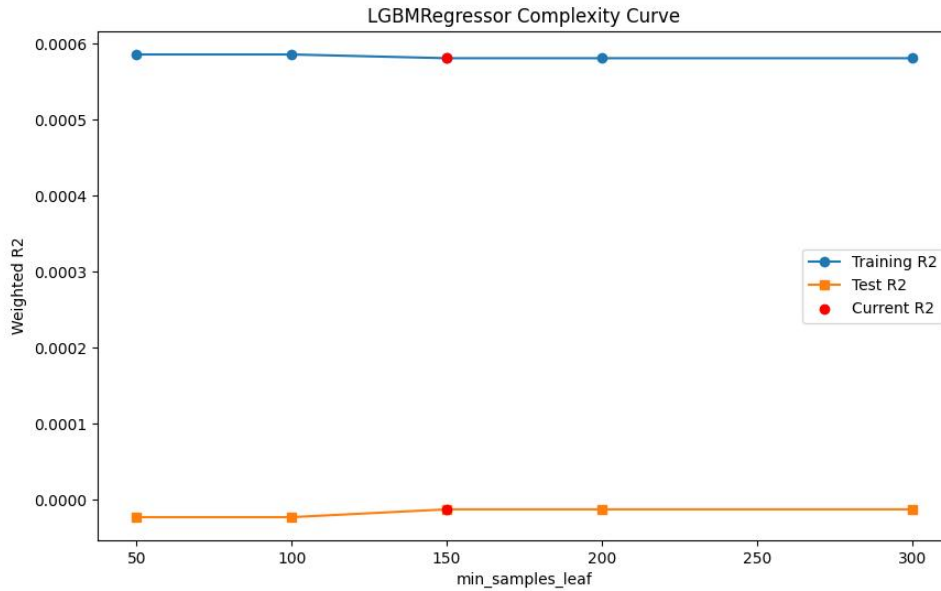


Figure 15: LightGBM Training vs Test Weighted R^2

4 Final Model - Performance and Results

For the final model, we selected XGBoost as it has the highest validation score, and exhibit an increasing followed by decreasing R^2 score. This pattern is desirable as it shows the model is less likely to overfit to the training data, and hence more likely to generalize to unseen test data.

4.1 30 day moving average correlation

The following plots present a 30 day moving average prediction of correlation against our target variable for various features over time. Interestingly, non of the features are sticking at 0, which implies that the features entail information.



Figure 16: Rolling Correlation for 30 days for different features

4.2 Distribution of Predictions

The distribution of each year's prediction is shown below in Figure 17. We observe that the predictions are roughly symmetric around zero for all years. For the years 2010 through 2013, the histograms show a very relatively narrow distribution centered around 0. In contrast,

the histogram for the year 2014 shows a similar narrow distribution, but it is centered around a slightly negative prediction value.

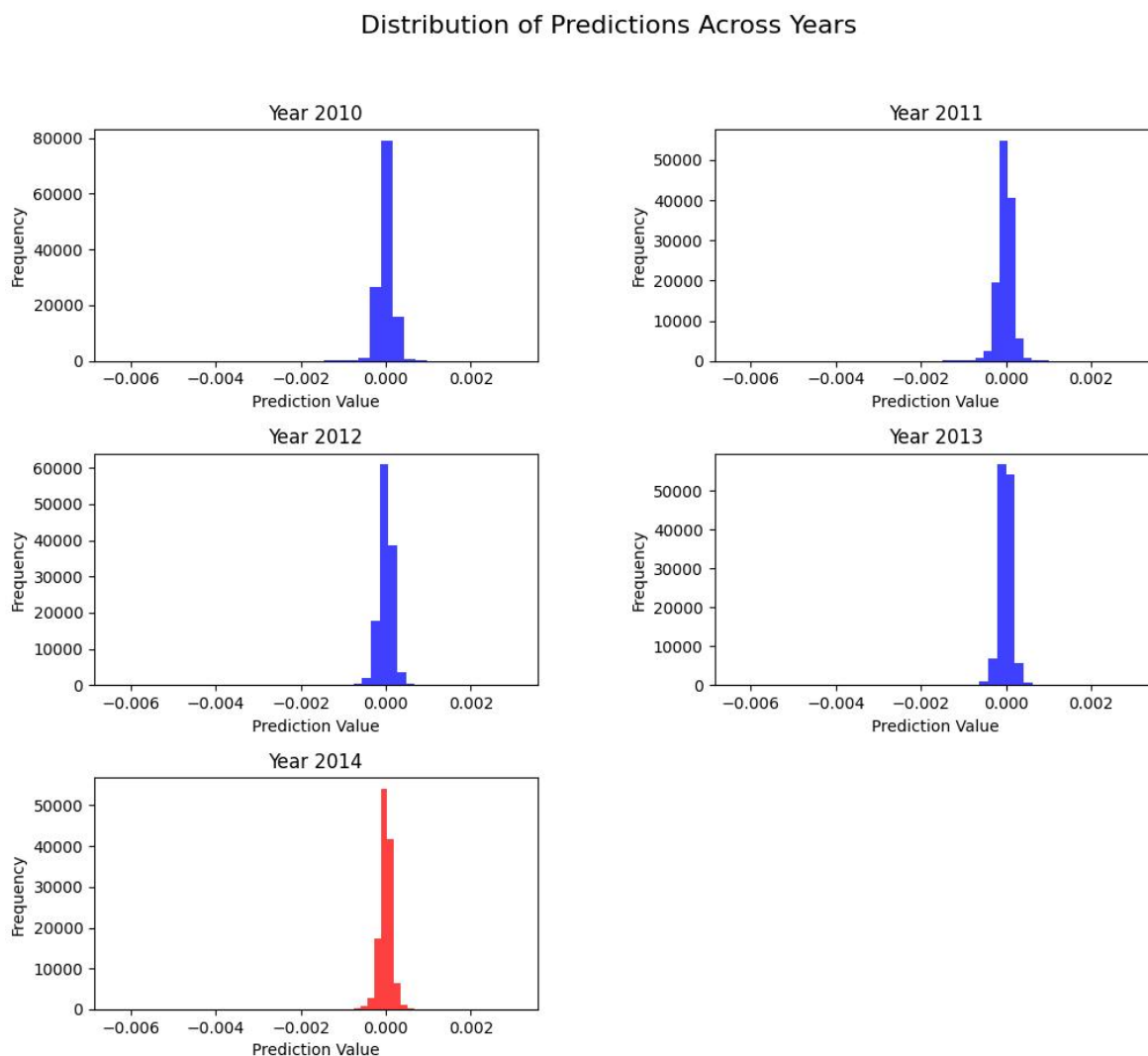


Figure 17: Yearly Prediction Distribution

4.3 Bin plots for returns

Bin plots were constructed to examine the distribution of financial returns over various years. These visualizations, as presented in the appendix, display the range of outcomes within each bin. Notably, some bins are characterized by negative outliers, indicating instances of particularly low returns. Conversely, other segments of the plots reveal positive

deviations from the median, suggesting occasional above-average returns. No seasonality patterns in the data over the years could be found.

A Bin Plots

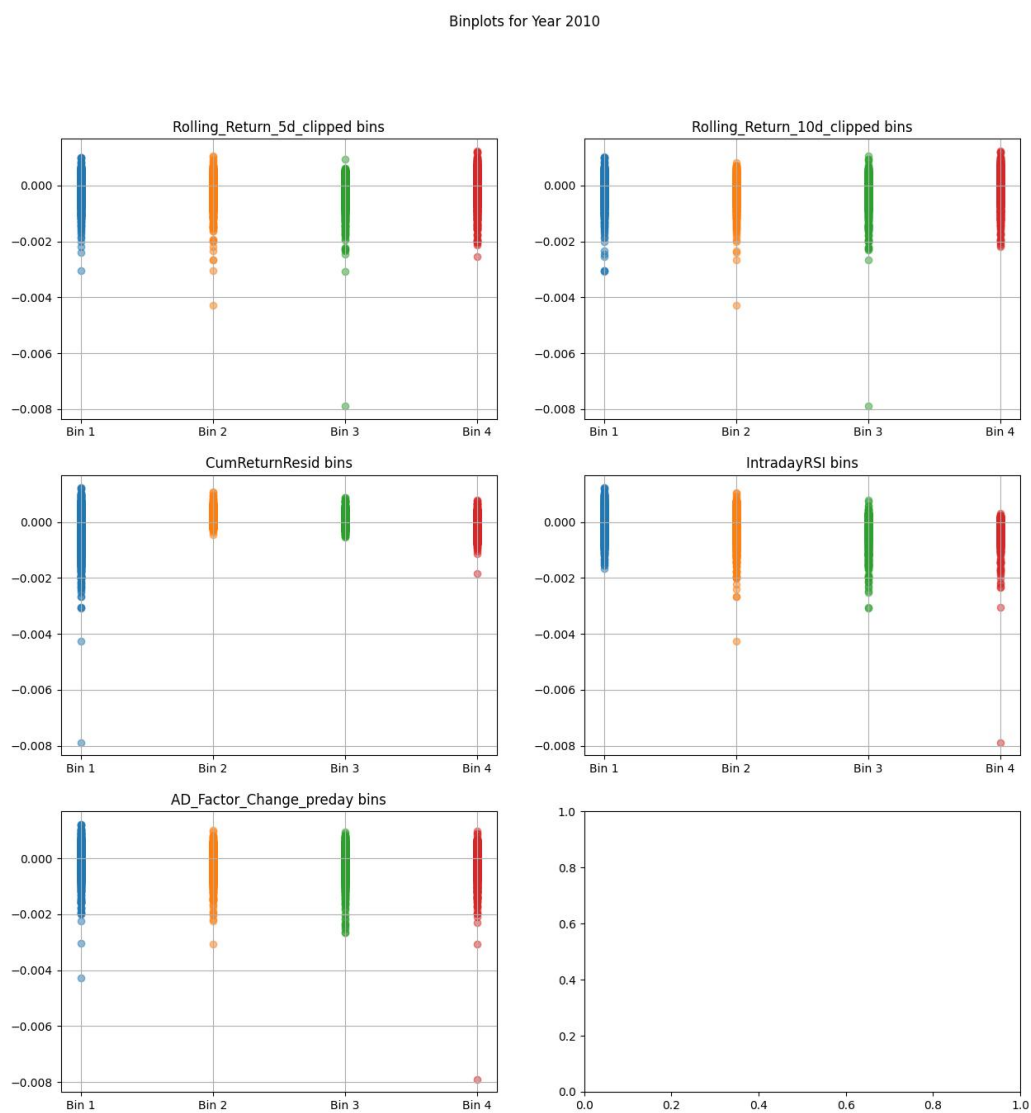


Figure 18: Bin plot for 2010

Binplots for Year 2011

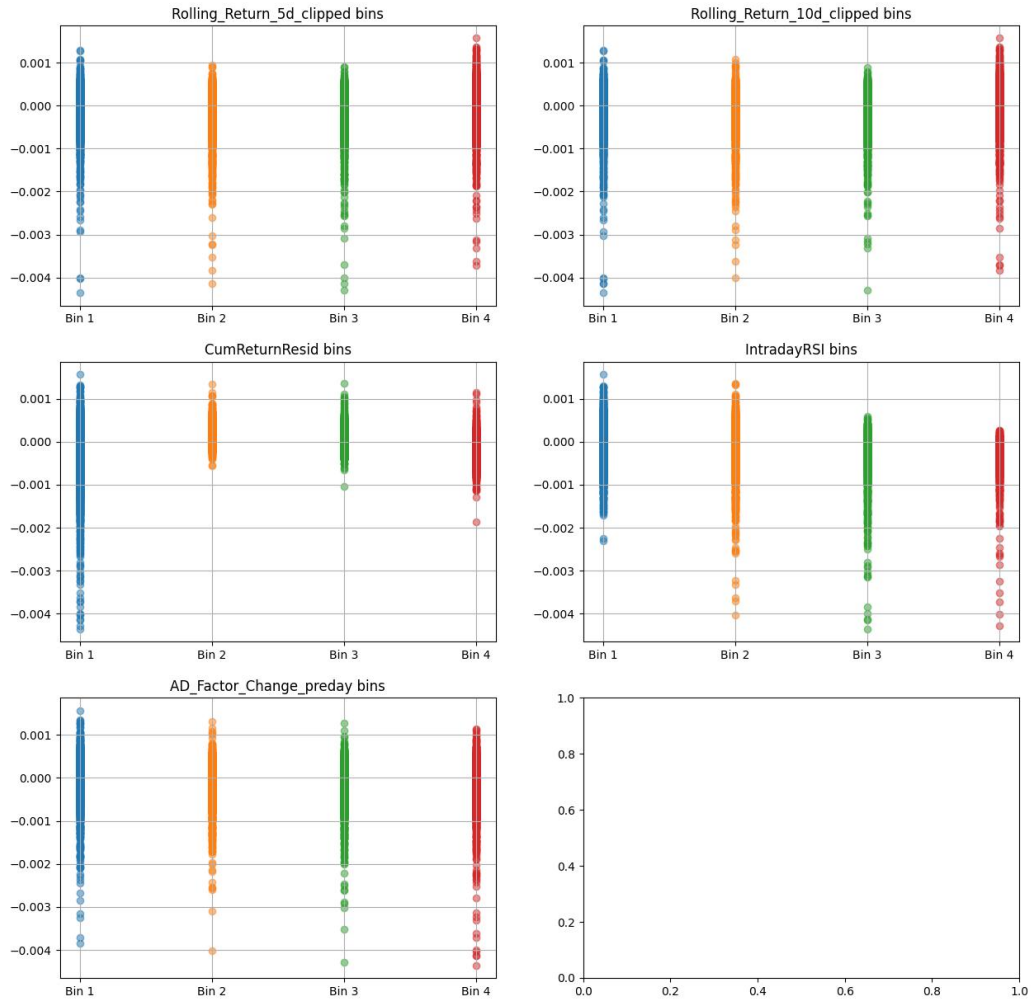


Figure 19: Bin plot for 2011

Binplots for Year 2012

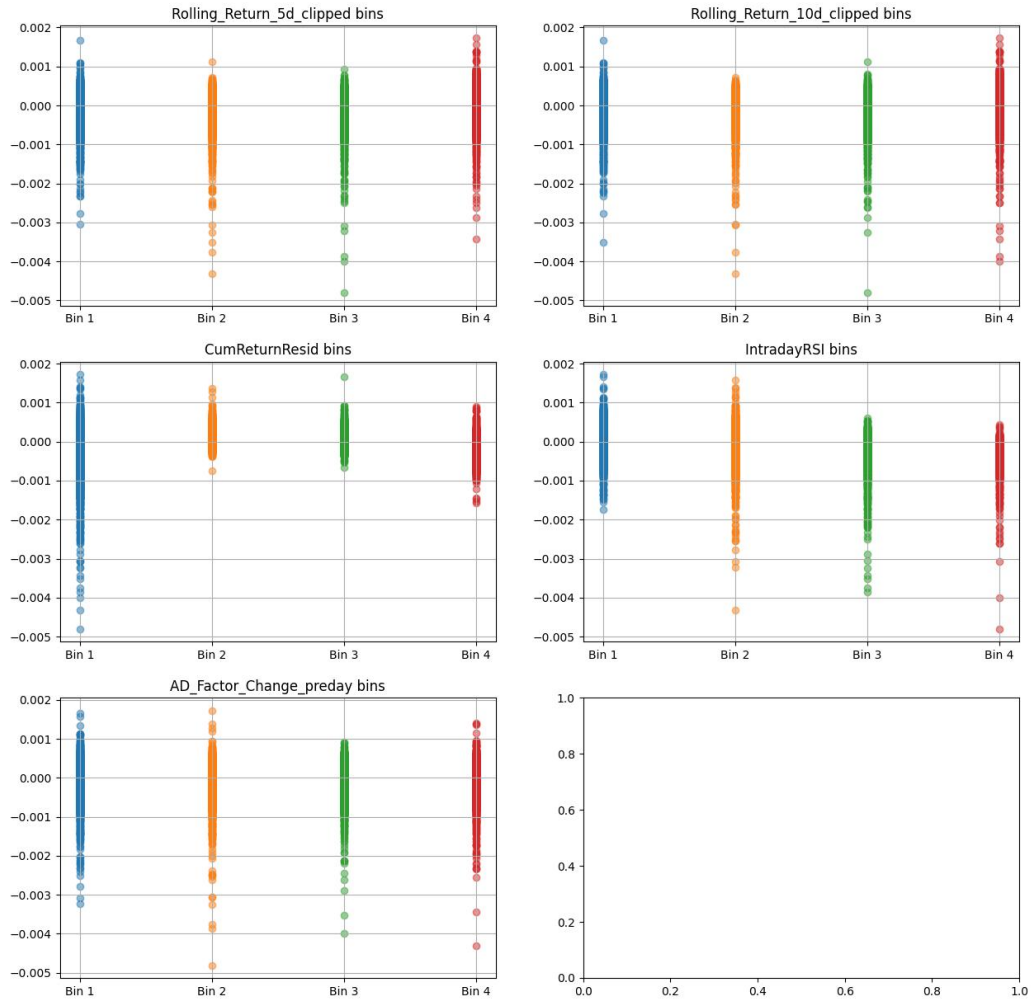


Figure 20: Bin plot for 2012

Binplots for Year 2013

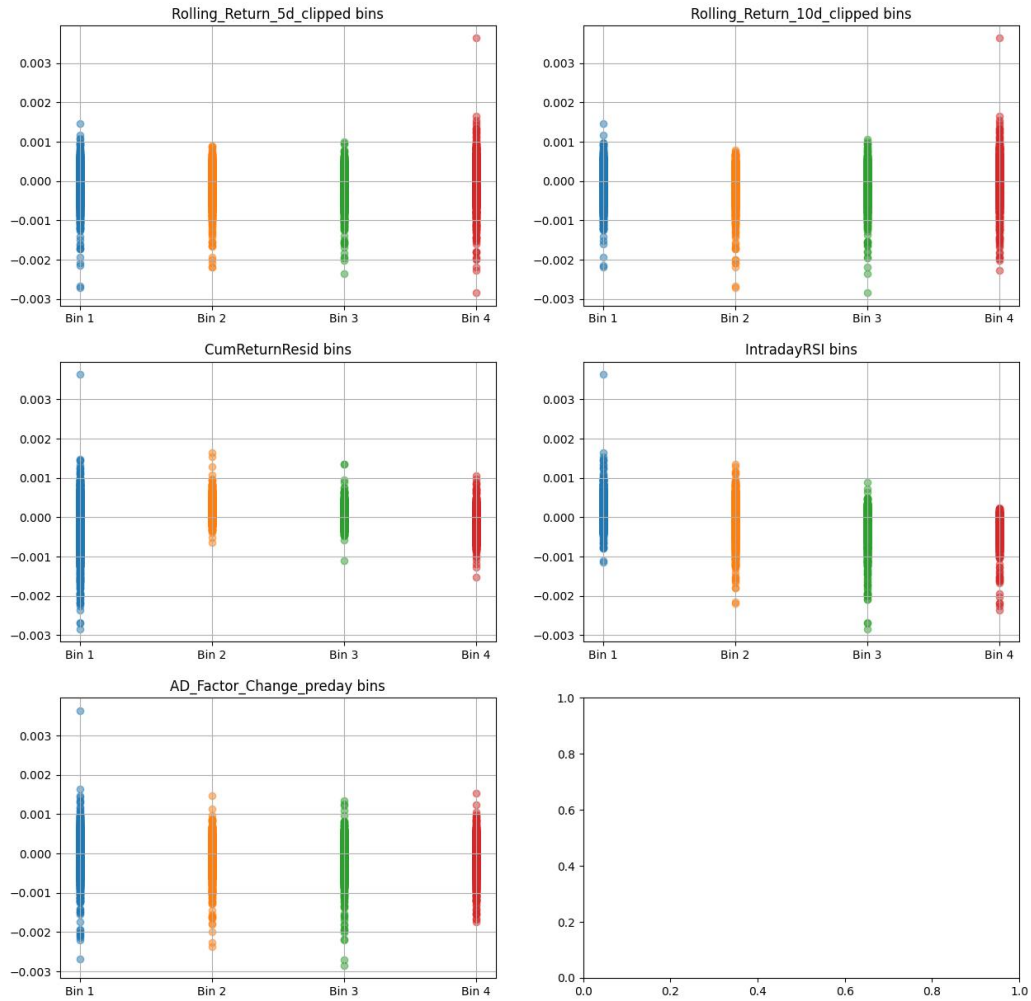


Figure 21: Bin plot for 2013

Binplots for Year 2014

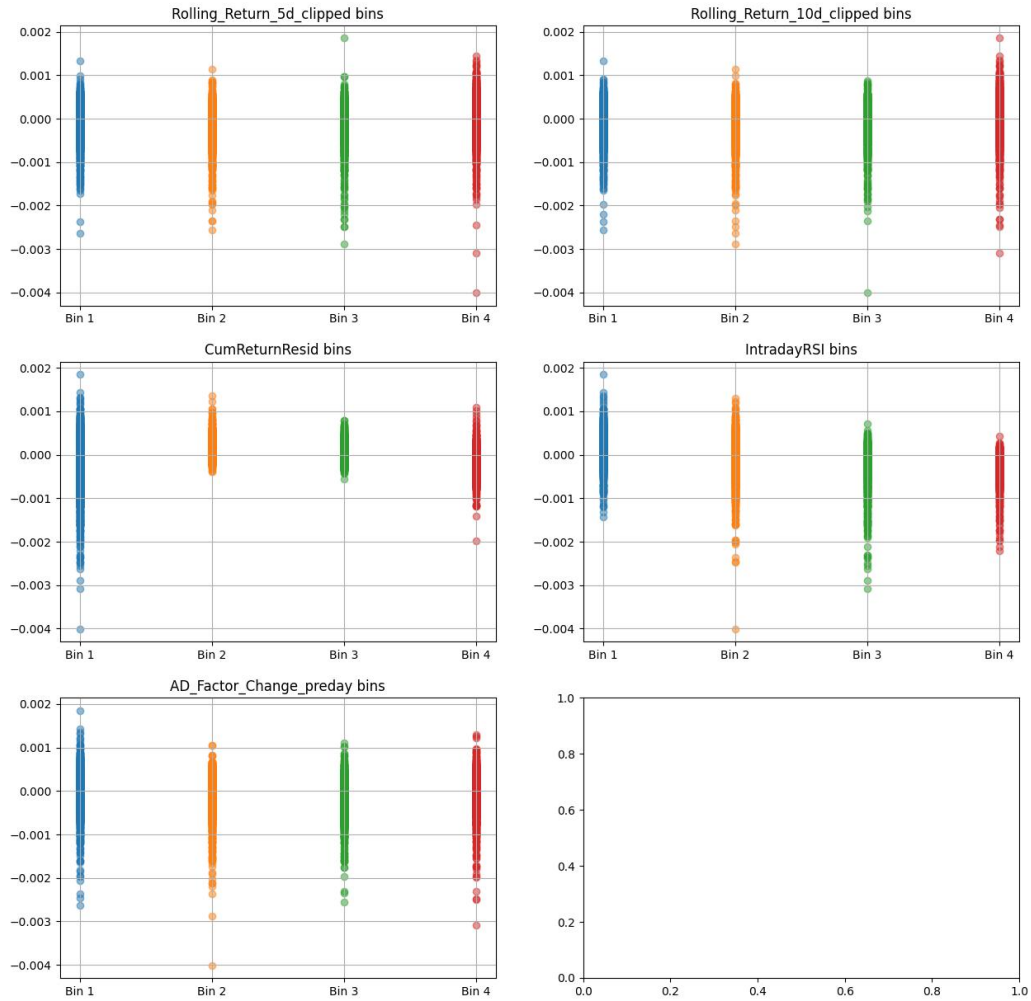


Figure 22: Bin plot for 2014